

Como está a solução e quanto o NLP acrescenta às regras

Resumo de 29/09/2026 · branch **lucas** · métrica oficial `kaggle_metric.py` (máximo 1,10) · 39 conjuntos de teste

A solução encontra citações de jurisprudência e de legislação em peças jurídicas e diz, para cada uma, se ela é **real** (existe na base oficial, com o identificador), **inventada** (completa, mas não existe) ou **incompleta** (faltam dados para identificar). Há duas versões: **só regras** e **regras + NLP**, que é a entregue.

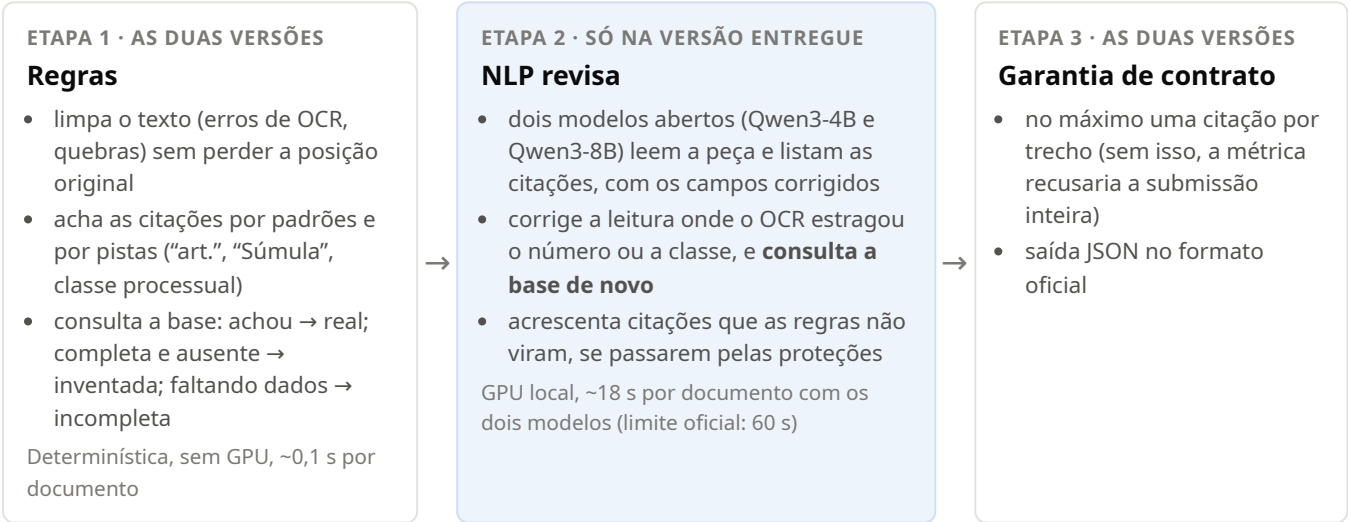
1,0999
no dev oficial, nas duas versões e na imagem Docker (máximo 1,10)

+0,035
na medida limpa (rodada 3: conjuntos inéditos, medidos uma vez, ao vivo): 1,0278 → 1,0627

+0,164
no pior caso, com metade dos caracteres corrompidos por OCR: 0,8706 → 1,0350

τ = 0
nos 39 conjuntos: nenhuma citação inventada saiu como real

Como funciona



O modelo nunca decide se uma citação é real. Ele só lê campos (tribunal, classe, número, artigo...). Quem decide é sempre a mesma consulta determinística à base oficial, igual à da versão só regras.

O que o NLP pode fazer

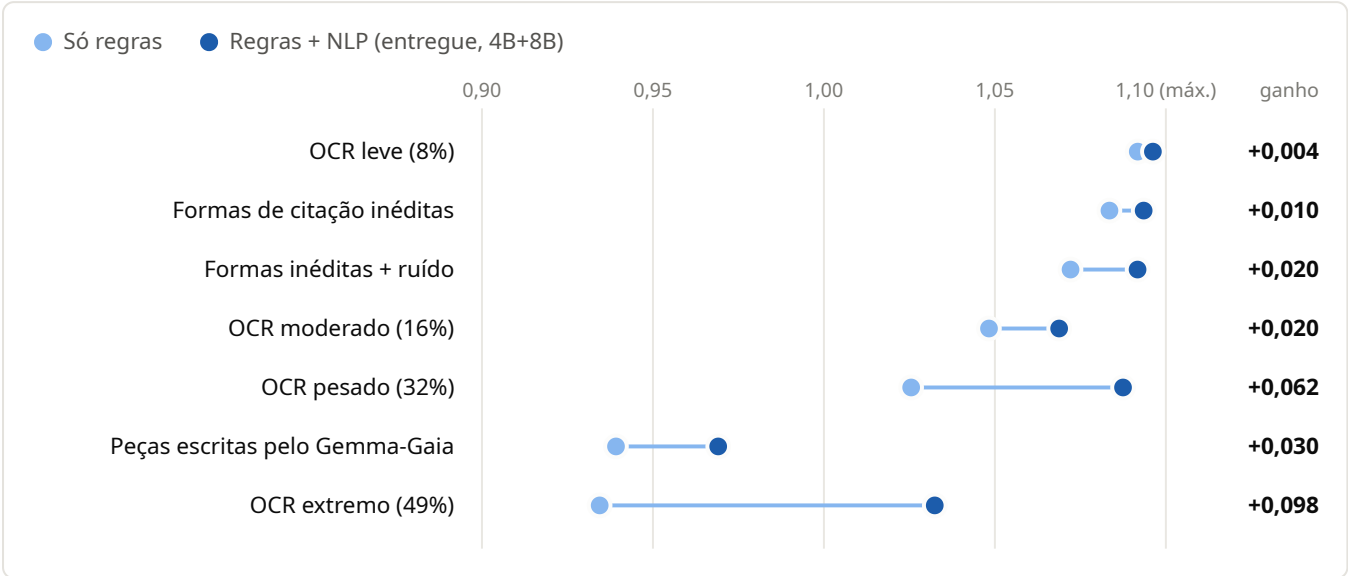
- ler um número ilegível por OCR ("1.570.531" → 1570531)
- encontrar citações escritas de um jeito que nenhum padrão previu
- recortar uma citação que as regras capturaram grande demais
- com dois modelos: juntar o que cada um encontrou

O que ele não pode fazer (proteções)

- mudar uma citação que as regras já resolveram como real
- usar um dígito que não esteja no próprio trecho da citação
- afirmar tribunal, classe ou lei sem pista no texto
- decidir quando os dois modelos discordam: vale o resultado das regras

A medida limpa: rodada 3

Sete conjuntos gerados **depois** de congelar o código, nunca usados para decidir nada e medidos **uma única vez**, com os modelos rodando de verdade. Quanto mais difícil o texto, maior o ganho do NLP.



Médias da rodada 3: só regras 1,0278 · só o 4B 1,0584 · só o 8B 1,0539 · os dois juntos (entregue) 1,0627. A rodada 2, medida do mesmo jeito em outros 8 conjuntos, deu 1,0418 → 1,0735: o ganho se repete.

Por tipo de texto: todos os 39 conjuntos

TIPO DE TEXTO	CONJUNTOS	SÓ REGRAS	ENTREGUE	GANHO	PIOR CONJUNTO
Formato oficial (como o dev)	6	1,0987	1,0996	+0,001	1,0995
Formas de citação inéditas	6	1,0766	1,0927	+0,016	1,0864
Armadilhas e texto poluído	6	1,0938	1,0951	+0,001	1,0892
Peças escritas por outros LLMs	6	0,9936	1,0381	+0,044	0,9691
OCR leve (8% dos caracteres)	3	1,0897	1,0974	+0,008	1,0960
OCR moderado (16%)	4	1,0596	1,0819	+0,022	1,0688
OCR pesado (32%)	4	1,0045	1,0641	+0,060	1,0413
OCR extremo (49%)	4	0,9465	1,0368	+0,090	1,0324
Todos	39	1,0484	1,0763	+0,028	

Médias por tipo; “pior conjunto” é a nota entregue no conjunto mais difícil do grupo. Rodadas 2 e 3: medição única ao vivo. Demais conjuntos: código final com as respostas guardadas do modelo (diferença medida entre ao vivo e guardado: até 0,004). No pior caso, o NLP recupera 72% do que o ruído tirou. Tabela completa no apêndice.

Onde está o ganho

- **Texto limpo, formato oficial:** quase nenhum. As regras já estão no teto; no dev, o NLP não muda nenhuma citação.
- **Formas novas e texto poluído:** ganhos pequenos e constantes.
- **Redação livre de outros modelos de linguagem:** ganho médio a alto; é o tipo de texto mais difícil.
- **OCR ruim:** o maior ganho, crescendo com o ruído.

O que o segundo modelo (8B) acrescenta

- sobre o 4B sozinho: +0,002 na média; acima em 16 conjuntos, igual em 22, abaixo em 1
- maiores ganhos: OCR extremo (3000) (+0,022), OCR extremo (9900) (+0,013), OCR moderado (3000) (+0,011)
- ajuda sobretudo nos textos mais difíceis; custa o dobro de tempo (~9 → ~18 s por documento), dentro do limite
- abaixo do 4B: peças do Qwen2.5 (iteração) (uma citação; limitação registrada no protocolo)

Execução e regras da competição

- modelos abertos do Hugging Face, com revisão fixa e sha256 no manifesto (Apache-2.0, sem fine-tune)
- contêiner sem internet, sem acesso a serviços do Ollama; modelos e base montados de fora, nunca dentro da imagem
- a mesma imagem serve a GPUs NVIDIA e AMD; os dois modelos juntos usam ~12 GB de VRAM

- se algo falhar (pesos ausentes, GPU, tempo), a execução segue sozinha na versão só regras
- orçamento próprio: se a média passar de 40 s por documento ou o total de 3 h, a camada se desliga
- decodificação sem aleatoriedade (temperatura 0, semente fixa)

Como medimos sem overfitting

- decisões tomadas só em conjuntos de iteração; holdouts gerados depois de congelar o código (com hash registrado)
- cada holdout medido uma única vez, ao vivo, sem olhar os erros; autores de texto e sementes de ruído nunca usados na iteração
- três rodadas independentes com o mesmo resultado: o NLP ganha das regras com $\tau = 0$

Situação em 29/09

FEITO	Dev 1,0999 com contrato validado; imagem só regras verificada no Docker, sem rede (1,0999).
FEITO	Imagem com NLP verificada de ponta a ponta na AMD, com os dois modelos: 1,0999 em duas execuções (diferença 0,0000) e 1,0999 no fallback sem pesos; 18,5 s por documento.
FEITO	Holdouts das rodadas 1, 2 e 3 medidos uma vez cada; benchmark dos dois modelos em 39 conjuntos; $\tau = 0$ em todos.
FEITO	Corrigido: garantia contra citações duplicadas, o segundo modelo que ficava fora da imagem e o acesso do Ollama à internet.
PENDENTE	Teste da imagem numa GPU NVIDIA (com um colega) e nova verificação com a imagem final.
PENDENTE	Tornar a busca da base robusta ao caminho em que a organização montar o arquivo; commit e push.

Apêndice: todos os conjuntos

CONJUNTO	ORIGEM	SÓ REGRAS	4B	8B	4B+8B (ENTREGUE)	τ
Formato oficial (como o dev)						
dev	dev oficial · Docker ao vivo	1,0999	1,0999	1,0999	1,0999	0
v5_iter	iteração	1,0979	1,0995	1,0987	1,0995	0
v5_iter_ruido	iteração	1,0973	1,0995	1,0995	1,0995	0
v5_iter_denso	iteração	1,0988	1,0996	1,0996	1,0996	0
denso	iteração	1,0995	1,0995	1,0995	1,0995	0
val	iteração	1,0987	1,0996	1,0996	1,0996	0
Formas de citação inéditas						
v5_holdout	iteração	1,0801	1,0957	1,0968	1,0968	0
v5_holdout_ruido	iteração	1,0714	1,0845	1,0854	1,0864	0
v5_holdout2	rodada 2 · ao vivo	1,0775	1,0936	1,0959	1,0954	0
v5_holdout2_ruido	rodada 2 · ao vivo	1,0750	1,0912	1,0908	1,0925	0
v5_holdout3	rodada 3 · ao vivo	1,0835	1,0926	1,0926	1,0935	0
v5_holdout3_ruido	rodada 3 · ao vivo	1,0721	1,0882	1,0929	1,0917	0
Armadilhas e texto poluído						
armadilhas	iteração	1,0985	1,0985	1,0985	1,0985	0
poluído	iteração	1,0926	1,0926	1,0936	1,0936	0
ineditos	iteração	1,0971	1,0996	1,0996	1,0996	0
ineditos_poluído	iteração	1,0925	1,0934	1,0925	1,0934	0
ineditos_v3	iteração	1,0955	1,0962	1,0972	1,0962	0
ineditos_v3_poluído	iteração	1,0868	1,0885	1,0892	1,0892	0
Peças escritas por outros LLMs						
llm_iter	iteração	1,0210	1,0241	1,0237	1,0191	0
llm_holdout	iteração	0,9999	1,0821	1,0821	1,0821	0
llm_llama_iter	iteração	0,9255	0,9963	0,9963	1,0056	0
llm_llama_holdout	rodada 2 · ao vivo	1,0324	1,0691	1,0691	1,0691	0
llm_gaia_holdout	rodada 2 · ao vivo	1,0435	1,0834	1,0834	1,0834	0
llm_gaia_holdout3	rodada 3 · ao vivo	0,9392	0,9614	0,9395	0,9691	0
OCR leve (8% dos caracteres)						
OCR leve (7700)	iteração	1,0829	1,0960	1,0960	1,0960	0
OCR leve (8800)	rodada 2 · ao vivo	1,0945	1,0999	1,0999	1,0999	0
OCR leve (9900)	rodada 3 · ao vivo	1,0918	1,0962	1,0950	1,0962	0
OCR moderado (16%)						
OCR moderado (3000)	iteração	1,0626	1,0843	1,0955	1,0955	0
OCR moderado (7700)	iteração	1,0658	1,0907	1,0887	1,0927	0
OCR moderado (8800)	rodada 2 · ao vivo	1,0619	1,0705	1,0665	1,0705	0
OCR moderado (9900)	rodada 3 · ao vivo	1,0483	1,0631	1,0660	1,0688	0
OCR pesado (32%)						
OCR pesado (3000)	iteração	1,0074	1,0700	1,0590	1,0700	0
OCR pesado (7700)	iteração	1,0071	1,0576	1,0557	1,0576	0
OCR pesado (8800)	rodada 2 · ao vivo	0,9779	1,0322	1,0404	1,0413	0
OCR pesado (9900)	rodada 3 · ao vivo	1,0255	1,0875	1,0801	1,0875	0
OCR extremo (49%)						
OCR extremo (3000)	iteração	0,8706	1,0133	0,9890	1,0350	0
OCR extremo (7700)	iteração	1,0094	1,0444	1,0300	1,0444	0
OCR extremo (8800)	rodada 2 · ao vivo	0,9717	1,0355	1,0246	1,0355	0
OCR extremo (9900)	rodada 3 · ao vivo	0,9344	1,0198	1,0109	1,0324	0

Fontes: docs/benchmarks/rodada3_conjunto.json e docs/benchmarks/final_ao_vivo_*.json (medições únicas ao vivo) e docs/benchmarks/tabela_codigo_atual.json (código final, respostas guardadas). Protocolo completo em tools/PROTOCOLO_NLP.md.